

DDL(Data Definition Language)

Command	Use
CREATE	Creates database or table
ALTER	Modifies table structure
DROP	Deletes table or database permanently
TRUNCATE	Removes all records but keeps structure
RENAME	Renames an existing table

1. CREATE Command

Used to create **database or table**

Syntax

```
CREATE DATABASE database_name;
```

```
CREATE TABLE table_name (
```

```
    column1 datatype,
```

```
    column2 datatype,
```

```
    ...
```

```
);
```

Example

```
CREATE DATABASE College;
```

```
CREATE TABLE Student (
```

```
    RollNo INT PRIMARY KEY,
```

```
    Name VARCHAR(50),
```

```
    Age INT,
```

```
    DOB DATE
```

```
);
```

2. ALTER Command

Used to **modify table structure**

Syntax

```
ALTER TABLE table_name ADD column datatype;
```

```
ALTER TABLE table_name MODIFY column datatype;
```

```
ALTER TABLE table_name DROP column;
```

Example

```
ALTER TABLE Student ADD Email VARCHAR(50);
```

```
ALTER TABLE Student MODIFY Age INT(3);
```

```
ALTER TABLE Student DROP Email;
```

3. DROP Command

Used to delete **table or database permanently**

Syntax

```
DROP TABLE table_name;
```

```
DROP DATABASE database_name;
```

Example

```
DROP TABLE Student;
```

```
DROP DATABASE College;
```

4. TRUNCATE Command

Deletes **all records**, table structure remains

Syntax

```
TRUNCATE TABLE table_name;
```

Example

```
TRUNCATE TABLE Student;
```

5. RENAME Command

Used to rename a table

Syntax

```
RENAME TABLE old_name TO new_name;
```

Example

```
RENAME TABLE Student TO Student_Details;
```

Practice Queries (DDL Commands)

1. Create a database named **University**.
2. Create a table **Employee** with columns:
 - EmpID (INT, Primary Key)
 - Name (VARCHAR)
 - Salary (DECIMAL)
 - DOJ (DATE)
3. Add a column **Email** to Employee table.
4. Change Salary column datatype to DECIMAL(12,2).
5. Delete the Email column from Employee table.
6. Rename table **Employee** to **Employee_Details**.
7. Remove all records from Employee_Details table without deleting its structure.
8. Delete the Employee_Details table completely.

DML COMMANDS

DML (Data Manipulation Language) commands are used to **insert, update, and delete data** in tables.

Command	Use
INSERT	Adds new records into table
UPDATE	Modifies existing records
DELETE	Removes records from table

1. INSERT Command

Used to **add new records** into a table

Syntax

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

or

```
INSERT INTO table_name (column1, column2)
```

```
VALUES (value1, value2);
```

Example

```
INSERT INTO Student VALUES (1, 'Amit', 20, '2004-05-10');
```

```
INSERT INTO Student (RollNo, Name, Age)
```

```
VALUES (2, 'Neha', 21);
```

2. UPDATE Command

Used to **modify existing records**

Syntax

```
UPDATE table_name
```

```
SET column = value
```

```
WHERE condition;
```

Example

```
UPDATE Student
```

```
SET Age = 22  
WHERE RollNo = 1;  
UPDATE Student  
SET Name = 'Rahul'  
WHERE RollNo = 2;
```

3. DELETE Command

Used to **remove records** from a table

Syntax

```
DELETE FROM table_name  
WHERE condition;
```

Example

```
DELETE FROM Student  
WHERE RollNo = 2;  
DELETE FROM Student;
```

(Deletes all records but keeps structure)

DELETE vs TRUNCATE

DELETE	TRUNCATE
DML Command	DDL Command
WHERE allowed	WHERE not allowed
Can be rolled back	Cannot be rolled back
Slower	Faster

Practice Queries (DML Commands)

1. Insert 5 records into **Student** table.
2. Insert only **RollNo and Name** for a student.
3. Update the **Age** of student whose RollNo is 3.
4. Change the **Name** of all students to uppercase.
5. Delete the record of student whose RollNo is 5.
6. Delete all records from Student table without deleting the table.
7. Increase age of all students by 1 year.

DQL COMMAND

DQL (Data Query Language) is used to **retrieve data** from a database.

SELECT Command

Syntax

SELECT column1, column2

FROM table_name

WHERE condition;

Command	Use	Syntax
SELECT	Retrieves data from table	SELECT column FROM table;
CLAUSES		
WHERE	Filters records	SELECT * FROM table WHERE condition;
DISTINCT	Removes duplicate values	SELECT DISTINCT column FROM table;
ORDER BY	Sorts result	SELECT * FROM table ORDER BY column ASC/DESC;
LIKE	Searches pattern	SELECT * FROM table WHERE column LIKE 'A%';

Command	Use	Syntax
LIMIT	Limits number of rows	SELECT * FROM table LIMIT n;
GROUP BY	Groups rows	SELECT col, COUNT(*) FROM table GROUP BY col;
HAVING	Filters groups	SELECT col, COUNT(*) FROM table GROUP BY col HAVING condition;

Examples

1. Select All Records

Use: Used to retrieve data from a table.

```
SELECT * FROM Student;
```

2. Select Specific Columns

```
SELECT Name, Age FROM Student;
```

3. WHERE

Use: Used to filter records based on a condition.

```
SELECT * FROM Student WHERE Age > 20;
```

4. DISTINCT

Use: Used to remove duplicate values from result.

```
SELECT DISTINCT Age FROM Student;
```

5. ORDER BY

Use: Used to sort records in ascending or descending order.

```
SELECT * FROM Student ORDER BY Age DESC;
```

6. LIKE

Use: Used to search data using pattern matching.

```
SELECT * FROM Student WHERE Name LIKE 'A%';
```

7. LIMIT

Use: Used to restrict number of records displayed.

```
SELECT * FROM Student LIMIT 3;
```

8. GROUP BY

Use: Used to group rows having same values.

```
SELECT Age, COUNT(*) FROM Student GROUP BY Age;
```

9. HAVING

Use: Used to filter grouped records.

```
SELECT Age, COUNT(*)  
FROM Student  
GROUP BY Age  
HAVING COUNT(*) > 1;
```

GROUPBY vs HAVING

• GROUP BY

Use:

- To **group rows that have the same value** in a column
- Usually used with **aggregate functions** like COUNT, SUM, AVG

Example Table: Student

Name	Course	Age
Amit	BTech	20
Neha	BTech	21

Name	Course	Age
Rahul	BSc	22
Sita	BSc	21
Rani	BTech	22

Query: Count students in each course

```
SELECT Course, COUNT(*) AS TotalStudents
FROM Student
GROUP BY Course;
```

Result:

Course TotalStudents

BTech 3

BSc 2

- GROUP BY groups all rows with the same **Course**
- Then COUNT(*) calculates **how many students** are in each group

HAVING

Use:

- Filters **groups** created by GROUP BY
- Like WHERE, but WHERE **cannot filter aggregated data**, HAVING can

Query: Show courses with more than 2 students

```
SELECT Course, COUNT(*) AS TotalStudents
FROM Student
GROUP BY Course
HAVING COUNT(*) > 2;
```

Result:

Course TotalStudents

BTech 3

- First, GROUP BY creates groups by **Course**
- HAVING applies condition on the **aggregated result** (TotalStudents > 2)
- Only BTech satisfies the condition

Practice Queries (DQL)

1. Display all records from Student table.
2. Display only Name and Age of students.
3. Display students whose age is greater than 19.
4. Display students whose name starts with 'A'.
5. Display all students in descending order of age.
6. Count total number of students.
7. Find average age of students.
8. Display age-wise student count.
9. Display only those ages having more than one student.
10. Display first 2 records from the table.

DCL(Data Control Language)

- **DCL commands** are used to **control access and permissions** in the database.
- GRANT gives rights, REVOKE removes rights.

Command	Use	Syntax	Example
GRANT	Gives permission to a user	GRANT privilege ON table TO user;	GRANT SELECT, INSERT ON Student TO user1;
REVOKE	Removes permission from a user	REVOKE privilege ON table FROM user;	REVOKE INSERT ON Student FROM user1;

TCL(Transaction Control Language)

TCL commands are used to **manage transactions** in a database.

1. **COMMIT** → makes all changes permanent
2. **ROLLBACK** → undoes changes to last COMMIT or SAVEPOINT
3. **SAVEPOINT** → partial rollback point inside a transaction

Command	Use	Syntax	Example
COMMIT	Saves all changes permanently	COMMIT;	After inserting records into Student, use COMMIT;
ROLLBACK	Undo changes in a transaction	ROLLBACK;	After a wrong UPDATE, ROLLBACK; will revert changes
SAVEPOINT	Marks a point in transaction to rollback partially	SAVEPOINT sp_name;	SAVEPOINT sp1;

PRACTICE QUERIES

Problem Statement 1

1. Create a table Student with columns: RollNo, Name, Age, Course, Email.
 2. Insert 5 student records.
 3. Update the course of a student based on RollNo.
 4. Retrieve all students older than 20, sorted by Name.
 5. Count students in each course and display courses with more than 1 student.
 6. Grant SELECT and INSERT privileges to a new user.
 7. Commit changes.
 8. Delete a student based on RollNo and rollback if deleted by mistake.
-

Problem Statement 2

1. Create table Employee with columns: EmpID, Name, Salary, DOJ, Department.
 2. Insert 5 employee records.
 3. Update salary for employees in a specific department.
 4. Select employees with salary > 50,000 and limit to 3 records.
 5. Group employees by Department and find average salary, filter departments with average > 50,000.
 6. Grant SELECT, UPDATE to a manager user.
 7. Rollback any wrong salary update.
-

Problem Statement 3

1. Create table Books with columns: BookID, Title, Author, Copies, Genre.
2. Insert 6 book records.
3. Update the number of copies for a specific book.
4. Select all books of a particular genre, sorted by Title.
5. Count total books per author.
6. Grant SELECT, INSERT to librarian, then revoke INSERT.
7. Truncate table to remove all records.

Problem Statement 4

1. Create table CourseEnrollment with columns: StudentID, CourseID, EnrollmentDate, Status.
2. Insert 5 enrollment records.
3. Update enrollment status of a student.
4. Select enrollments after a specific date.
5. Group by CourseID and count students per course.
6. Use COMMIT to save changes and ROLLBACK to undo wrong status updates.
7. Drop table after completion.

Problem Statement 5

1. Create table Sales with columns: SaleID, Product, Quantity, Price, SaleDate.
2. Insert 6 sales records.
3. Update price of a product.
4. Select sales with Quantity > 10, sorted by SaleDate.
5. Group by Product and find total sales amount per product.
6. Grant SELECT, UPDATE to a sales manager.
7. Commit and rollback certain changes if required.